

PROJECT DOCUMENTATION

Katalon TestOps Automation Management Dashboard

An executive view of automation growth, execution health, stability, and project performance.

Prepared: August 16, 2026

Application: Private hosted dashboard

Primary source: katalon_testops_sample_dataset.csv

Technology: React/Vinext, TypeScript, Python, SQLite/D1

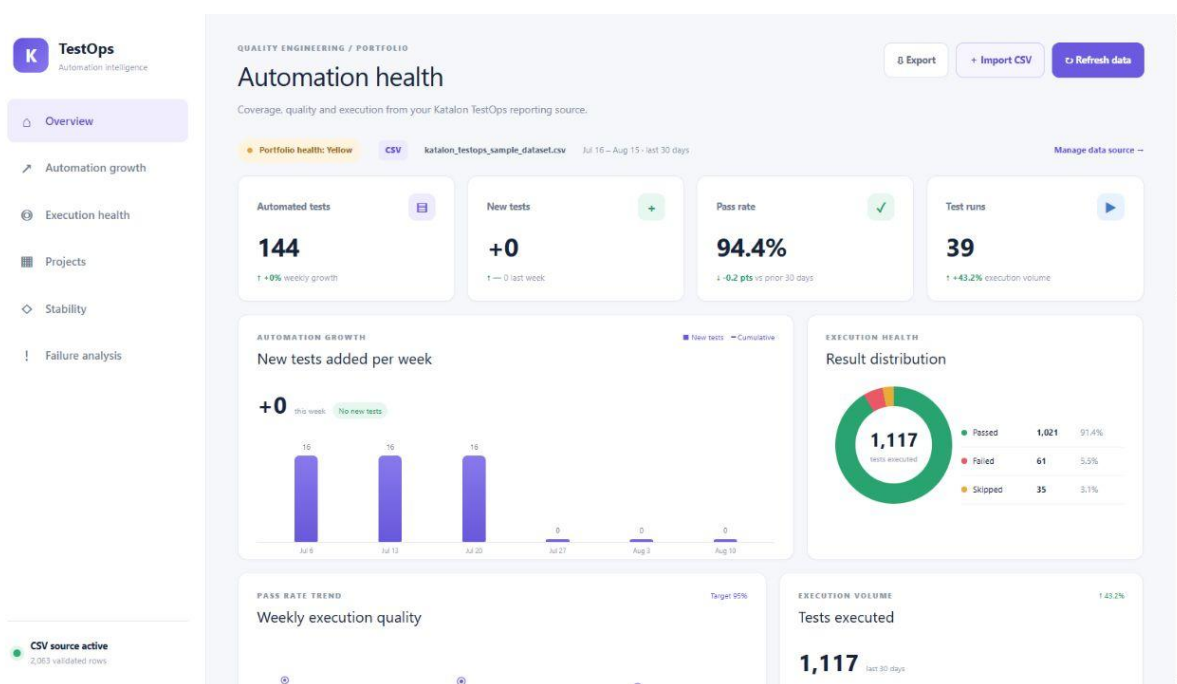


Figure 1. Live executive dashboard using the supplied CSV dataset.

The dashboard centralizes Katalon TestOps automation reporting across WebOps, Salesforce, BSP Services, and MyKC. It replaces manual weekly reporting with a management-level view that can answer how much automation exists, whether coverage is growing, how often tests run, which projects are healthy, and where stability requires attention.

- One portfolio view for QE leadership instead of project-by-project spreadsheets.
- Consistent definitions for pass rate, failure rate, growth, and flakiness.
- Historical weekly and monthly trends that support period-over-period conversations.
- Screenshot- and PDF-friendly presentation with low-level technical detail available through drill-down.
- Extensible ingestion: Katalon API connectivity plus repeatable monthly CSV imports.

Area	What was built
Executive summary	Portfolio health, automated tests, new tests, pass rate, runs, comparisons, and refresh state.
Growth	Weekly new-test bars, cumulative context, and project-level additions.
Execution health	Passed/failed/skipped distribution, weekly pass-rate trend, and execution volume.
Project performance	Per-project tests, execution outcomes, duration, status, and drill-down entry point.
Stability & failures	Flaky-test counts, repeat failures, top failing tests, and extensible failure categories.
Data management	API-ready integration, normalized analytics schema, snapshots, and CSV import with validation.
Sharing	Print/PDF export, responsive layout, private hosted URL, and social preview asset.

The active source is `katalon_testops_sample_dataset.csv`, supplied for the project and copied into the repository under `sources/`. A SHA-256 comparison confirmed the repository copy matches the supplied file.

Property	Observed value
Rows	2,063 execution-result records
Projects	4: BSP Services, MyKC, Salesforce, WebOps
Distinct automated tests	144
Distinct runs	96 across the full file
Execution date range	May 26 through August 15, 2026
Latest 30-day window	1,117 results; 1,021 passed; 61 failed; 35 skipped
Latest-window pass rate	94.4% using Passed / (Passed + Failed)
Stability signals	27 flaky tests and 27 repeat-failure rows in the latest window

The dataset contains execution and test identifiers, project and suite names, test creation timestamps, execution timestamps, environment, browser, status, duration, retry attempts, flakiness flags, repeat-failure indicators, failure category, agent name, and Katalon version. These fields support both executive aggregates and later drill-down.

- CSV files are validated before replacing the active source.
- Required fields include `execution_id`, `run_id`, `project_name`, `test_case_id`, `test_case_created_at`, `execution_started_at`, `status`, and `duration_seconds`.
- Optional flakiness and failure-category fields enrich the management view.
- Successful imports retain the original CSV and a calculated summary; the latest successful import is loaded on future visits.
- Invalid files never replace the current dataset.

The solution separates collection, storage, metric calculation, and presentation. The initial hosted application uses a React/Vinext interface and Cloudflare D1-compatible persistence, while the Python layer provides a reusable Katalon client and SQLite proof-of-concept path.

1. Katalon TestOps API or monthly CSV export
2. Defensive ingestion and validation layer
3. Normalized local analytics database and daily snapshots
4. Pure metric calculations with documented definitions
5. Dashboard API and executive React interface

Component	Responsibility	Representative assets
Integration	Authentication from environment variables, pagination, retry/backoff, shape discovery.	backend/katalon/
Analytics storage	Projects, tests, executions, results, daily metrics, CSV imports.	backend/database/, db/, drizzle/
Metrics	Pass/failure rates, growth, health thresholds, flakiness.	backend/metrics/, lib/csv-metrics.ts
Application API	Stores and retrieves monthly CSV imports.	app/api/imports/route.ts
Management UI	KPIs, trends, project health, failures, summary, imports.	app/Dashboard.tsx
Configuration	Projects, thresholds, refresh behavior, secret names.	config/projects.yaml, .env.example

- Pass rate excludes skipped and error statuses: Passed / (Passed + Failed).
- Duplicate execution records are prevented through execution IDs and composite result keys.
- Growth uses test creation timestamps; daily snapshots are the fallback when timestamps are unavailable.
- Reporting windows are anchored to the maximum execution date in the active dataset, not the machine clock.
- Failure category defaults to Unknown while retaining room for Product, Automation, and Environment classifications.

The build was driven through a sequence of natural-language prompts. The prompts below are concise records of the actual intent used during the workflow; the first prompt was substantially longer and supplied the complete metric, architecture, data-model, and UI specification.

Stage	Prompt / instruction used
Initial build	“Build a Katalon TestOps Automation Management Dashboard” followed by detailed requirements for executive KPIs, weekly growth, project metrics, execution trends, failures, flakiness, duration, thresholds, filters, comparisons, drill-down, SQLite/PostgreSQL-ready storage, snapshots, exports, architecture, and phased delivery.
Data-source change	“Ensure you use this as data source. Also, add a feature to import CSV as source for future months.”
Social preview	“Create a polished executive automation quality dashboard social card” matching the dashboard’s navy, violet, white, green, amber, and red visual system, with exact text “AUTOMATION HEALTH” and “95.5% PASS RATE.”
Documentation & handoff	“Create project documentation explaining what you built,” including datasets, vibe-coding prompts, iterations, learnings, a short demo video, and a GitHub link.

- Translated a broad management objective into a working application architecture and implementation plan.
- Generated the UI, CSV parser, metric engine, API route, database schema, migrations, tests, documentation, and deployment package.
- Inspected the supplied dataset to reconcile dashboard figures with source records.
- Iterated from compilation and runtime feedback rather than treating generated code as final.
- Captured and verified the live hosted interface for the demo and project handoff.

Iteration	What changed	Why it mattered
1. Executive visual prototype	Built the first management screen with representative KPIs, growth, distribution, trends, projects, stability, and summary.	Established information hierarchy and leadership-ready visual direction before live integration.
2. Integration foundation	Added environment-based credentials, paginated Katalon client, retries, normalized storage, snapshots, and tested metrics.	Separated API uncertainty from analytics logic and avoided hard-coded credentials or project IDs.
3. Source-backed dashboard	Replaced representative values with aggregates from the supplied 2,063-row CSV.	Made the management story traceable to an actual dataset.
4. Monthly import workflow	Added drag/drop CSV upload, required-column validation, 5 MB limit, recalculation, durable storage, and latest-import loading.	Made future reporting repeatable without changing application code.
5. Data reconciliation	Anchored the default 30-day window to the latest execution date, validated 144 tests and 94.4% pass rate, and added source labels.	Prevented misleading results when sample data dates differ from the current clock.
6. Production hardening	Added D1 migration, build/tests, responsive/print styling, private deployment, and documentation.	Converted a prototype into a shareable proof of concept.

- The first dependency setup path was not available in the Windows shell, so the equivalent bundled runtime path was used.
- A CSV summary helper initially referenced an undeclared prior-week variable; source-level verification caught and corrected it.
- Generated Python cache files were removed from version control and permanently ignored.
- The first dashboard used sample values; these were intentionally replaced once the real CSV was provided.

- Management dashboards benefit from a strong first-screen hierarchy: four KPIs, two trend panels, project health, and a plain-language summary were easier to interpret than a dense execution table.
 - Metric definitions must be explicit. A pass rate that includes skipped tests would have told a different story from the documented Passed / (Passed + Failed) definition.
 - Growth is impossible to reconstruct reliably from only a current test count. Test creation dates or daily snapshots are essential.
 - A reporting window should be anchored to the dataset's latest date for static or delayed exports; anchoring to today can create an empty dashboard.
 - CSV import is a practical bridge while TestOps endpoint shapes and tenant permissions are being validated. It also provides an auditable fallback for monthly leadership reporting.
-
- Large prompts are useful for establishing scope, but implementation quality improved when the work was divided into bounded slices: visual shell, integration, storage, metrics, live data, imports, and publishing.
 - AI-generated code still requires reconciliation against source data, builds, migrations, and targeted assertions. The most valuable verification was checking exact row counts and KPI totals.
 - A reusable parser and pure metric functions are safer than embedding calculations directly in UI components.
 - Clear file boundaries made later changes faster: swapping the data source did not require redesigning the dashboard.
 - The best vibe-coding workflow was conversational but evidence-driven: prompt, build, inspect, test, correct, and only then publish.
-
- Complete Phase 1 against the organization's live Katalon TestOps tenant and confirm endpoint/field mappings.
 - Add scheduled incremental synchronization and operational alerting for failed imports or stale data.
 - Add role-based access for import actions and an import-history screen with rollback.
 - Expand technical drill-down routes for suites, executions, and individual test cases.
 - Add server-generated Excel and management-report exports after live database queries are fully wired.

The final application was validated through a production Vinext build, source-backed CSV assertions, UI content tests, metric checks, D1 migration generation, and a private deployment. The supplied CSV reconciled to 2,063 rows, 144 distinct tests, four projects, 1,117 results in the latest 30-day window, and a 94.4% pass rate.

- How many automated tests exist and how many were added this week?
- Is execution volume increasing, and what is the overall pass rate?
- Which project performs best and which needs attention?
- Are failures, repeat failures, or flaky tests becoming a management concern?
- How long do executions take, and which projects are slower?
- What changed relative to the prior reporting period?
- Which source file is active, and can next month's data be imported safely?

Deliverable	Location
Live dashboard	https://katalon-automation-health.satyamlowhitha.chatgpt.site
Source dataset	<code>sources/katalon_testops_sample_dataset.csv</code>
Setup and technical guide	<code>README.md</code>
Project documentation	<code>deliverables/Katalon_Automation_Dashboard_Project_Documentation.docx</code>
Demo video	<code>deliverables/Katalon_Automation_Dashboard_Demo.mp4</code>